

以下是七个PDF中涉及的所有名词及其解释：

期末复习一：词法解析-语法解析LL(1)文法

名词	解释
词素 (Lexeme)	词法分析的基本单位，源程序中匹配某个模式的字符序列
词法单元 (Token)	词法分析器输出的抽象单元，由词素和附加信息组成
RE (正则表达式)	可以简洁、精确地描述词法单元的模式
DFA (确定有限自动机)	每个状态对每个输入符号只有唯一一个转换的自动机
NFA (非确定有限自动机)	一个状态对同一个输入符号可以有多个转换的自动机
ϵ -closure (ϵ 闭包)	从NFA某个状态集合出发，只通过 ϵ 转换能够到达的所有状态的集合
子集构造法	将NFA转换为DFA的算法，DFA每个状态对应NFA一个状态集
Thompson构造法	从正则表达式构造NFA的算法，按结构递归构造
左递归	文法中非终结符号A对某个串 α 存在推导 $A \Rightarrow A\alpha$
立即左递归	存在产生式 $A \rightarrow A\alpha$ 的左递归
自顶向下分析	从根结点开始向下到达叶子结点构造语法分析树的过程
递归下降分析	每个非终结符号对应一个过程的自顶向下分析方法
预测分析法	一种确定性的、无回溯的分析技术，通过查看下一个输入符号选择产生式
回溯	选择的产生式不匹配时退回到之前选择点尝试其他产生式
提取左公因子	消除文法中具有相同前缀的产生式的技术
FIRST(α)	可以从 α 推导得到的串的首符号的集合
FOLLOW(A)	可能在某些句型中紧跟在非终结符号A右边的终结符号的集合
LL(1)文法	第一个L自左向右扫描，第二个L产生最左推导，1向前看一个符号
预测分析表	二维表 $M[A, a]$ ，告诉分析器非终结符A、输入符号a时选择哪条产生式
文法	描述语言语法结构的形式规则系统
终结符号	文法中不能再被替换的基本符号
非终结符号	文法中可以被替换的符号
产生式	文法规则，形式为 $A \rightarrow \alpha$
推导	从开始符号出发反复应用产生式得到终结符号串的过程
最左推导	每一步都替换最左边的非终结符号的推导
句型	从开始符号推导得到的符号串（可包含非终结符号）

名词	解释
句子	只包含终结符号的句型

期末复习二：语法解析LR(1)文法

名词	解释
自底向上分析	将串 w 归约为文法的开始符号的过程，从叶子开始向上到达根结点
移入-归约	自底向上语法分析的重要通用框架，核心操作是移进与归约
归约	与某产生式体匹配的子串被替换为该产生式头部的非终结符号
移进	将输入中的下一个符号移入到栈中
最右推导	每一步都替换最右边的非终结符号的推导
句柄	与某产生式体匹配的子串，是当前最右句型中需要被归约的部分
移进-归约冲突	分析器无法决定是"移进"还是"归约"
归约-归约冲突	分析器无法决定"用哪条规则归约"
LR(k)分析	L最左扫描，R反向构造最右推导，k向前看k个符号
LR(0)项	文法的一个产生式加上在其产生式体中某处的一个点
规范LR(0)项集族	构建LR(0)自动机的基础
增广文法	G中增加新开始符号 S' 并加入产生式 $S' \rightarrow S$ 得到的文法
CLOSURE (项集闭包)	如果 $A \rightarrow \alpha \cdot B\beta$ 在CLOSURE(I)中，对B的任意产生式 $B \rightarrow \gamma$ ，将 $B \rightarrow \cdot \gamma$ 加入
GOTO(I,X)	I中所有形如 $[A \rightarrow \alpha \cdot X\beta]$ 所对应的项 $[A \rightarrow \alpha X \cdot \beta]$ 的集合的闭包
内核项	初始项 $S' \rightarrow \cdot S$ 以及所有点不在最左边的项
非内核项	除 $S' \rightarrow \cdot S$ 外点在最左边的项
SLR (Simple LR)	在LR(0)项集基础上加上FOLLOW集来控制归约动作
SLR(1)文法	根据SLR算法构造的分析表中没有冲突的文法
LR(1)文法	可以识别所有确定性上下文无关文法
LALR (Lookahead LR)	通过更精确的向前看信息来减少冲突
可归约流图	任何优先生成树中的所有后退边都是回边的流图

期末复习三：语义分析

名词	解释
语法制导定义 (SDD)	将语义规则附着到文法产生式上的形式系统，= 上下文无关文法 + 属性 + 语义规则
语法制导翻译 (SDT)	在产生式体中加入语义动作并在适当时执行
属性	与文法符号相关联的值，用来存储语义信息
属性文法	没有副作用的SDD
综合属性	自下而上计算，从子结点传到父结点
继承属性	自上而下或兄弟结点之间传递
S属性的SDD	只包含综合属性的SDD，可按自底向上方式求值
L属性的SDD	每个属性要么是综合要么是继承，且计算满足特定约束
依赖图	描述分析树上各属性实例之间的信息流
注释语法分析树	包含各结点各属性值的语法分析树
语义规则	抽象层次上的定义，描述属性如何传播、计算和约束
语义动作	实现层次上的指令，嵌入产生式中的可执行代码
抽象语法树 (AST)	去除冗余语法细节，只保留核心结构的树
具象语法树 (CST)	按文法规则推导得到的完整树，精确反映文法层次
符号表	记录源程序中各种符号的数据结构，本质是名称→属性的映射表
作用域	标识符有效的程序区域
符号表栈	支持作用域嵌套的设计，进入新作用域新建，退出则弹出
类型检查	语义分析阶段检查类型是否匹配的过程
常量折叠	编译时计算常量表达式的值的优化技术

期末复习四：中间代码生成

名词	解释
中间表示 (IR)	编译器在前端与后端之间使用的、与源语言和目标机器都相对独立的程序表示形式
高层IR	接近源语言结构，便于高层语义相关优化
中层IR	典型如三地址码，足够接近机器以便分析
低层IR	显式寄存器/栈、分支、加载/存储，易映射到目标指令
三地址码 (TAC)	每条指令最多涉及三个“地址”的中间表示，格式：result = arg1 op arg2
四元式	三地址码实现方式，格式(op, arg1, arg2, result)

名词	解释
三元式	三地址码实现方式，格式(op, arg1, arg2)，用位置引用结果
间接三元式	包含指向三元式的指针列表
静态单赋值形式 (SSA)	每个变量仅被赋值一次的中间表示，通过下标区分不同版本
Φ 函数	SSA中在控制流合流点合并不同路径定值的伪指令
堆栈式IR	操作数隐含在栈上的IR（如JVM字节码）
布尔表达式	将布尔运算符作用在布尔变量或关系表达式上构成的表达式
关系表达式	形式为 $E_1 \text{ rel } E_2$ 的表达式
短路求值	B_1
回填 (Backpatching)	先生成带占位符的跳转指令，后续再填充实际标签的延迟绑定技术
truelist	布尔表达式取值true时执行的跳转指令位置列表
falselist	布尔表达式取值false时执行的跳转指令位置列表
nextlist	语句执行完毕后紧接着执行的跳转指令位置列表
穿越 (Fall-through)	不生成跳转指令，顺序进入下一条指令
基本块	中间代码的最大直线指令序列，控制流只能从第一个指令进入
流图 (控制流图)	结点是基本块，边指明执行顺序的图

期末复习五：运行时环境

名词	解释
静态区	大小在编译时已知的数据对象的存放区域
栈区	存放活动记录的数据结构，栈向较低地址方向增长
堆区	存放生命周期不确定的数据对象的区域
活动记录 (栈帧)	过程调用和返回由控制栈管理，每个活跃活动对应栈中一个活动记录
活动树	表示程序运行过程中过程调用序列的树
控制栈	管理过程调用和返回的栈
栈底指针 (BP)	指向栈帧底部的寄存器
栈顶指针 (SP)	指向栈顶的寄存器
返回地址	函数调用时保存的调用点之后的指令地址
存储管理器	分配/回收堆区空间的子系统

名词	解释
碎片	堆区逐渐被割裂成空闲存储块和已用存储块的交错
窗口 (Hole)	堆中的空闲存储块
Best-Fit	将请求分配在满足请求的最小窗口中的策略
First-Fit	将对象放置在第一个能容纳请求的窗口中的策略
垃圾	不能被引用（不可达）的数据
垃圾回收	自动回收不可达数据的机制
可达性	存储块可以被程序访问到的性质
根集	不需要指针解引用就可以直接访问的数据
增变者	用户程序，修改堆中的对象集合
引用计数	每个对象有引用计数字段，为0则回收的垃圾回收方法
标记-清扫式垃圾回收	从根集标记可达对象，遍历堆释放不可达对象的算法
标记并压缩垃圾回收	在标记清扫基础上对可达对象重定位以消除碎片
拷贝回收器	堆分为两个半空间，回收时可达对象拷贝到另一半空间
半空间	拷贝回收器中堆被分成的两份相邻空间（fromspace和tospace）
分代回收	将堆区切割成块并标号，较年轻的世代更频繁被回收
记忆集	分代回收中存储高号堆块对低号堆块引用的集合
Stop-the-world	GC期间整个系统被挂起的现象
空闲空间列表	包含堆中所有未分配存储块的列表
边界标记	标识存储块空闲/已用状态和大小的标记

期末复习六：代码生成-寄存器分配

名词	解释
指令选择	把IR程序映射成为目标机上可运行代码序列的过程
指令代价	每个指令的固定代价，设定为1加上运算分量寻址模式的代价
寄存器分配	确定哪些变量值应该存放在寄存器中的过程
寄存器指派	确定哪个变量使用哪个具体寄存器的过程
加载指令 (LD)	把地址addr中的内容加载到寄存器
保存指令 (ST)	把寄存器中的内容保存到内存
MAXLIVE	每条指令处存活值的最大数量

名词	解释
溢出 (Spill)	寄存器不够用时将部分值保存到内存中的操作
后续使用信息	变量在后续指令中是否会被使用的信息
活跃变量	在程序执行完语句i后，x中存放的值将被后面的语句使用
寄存器描述符	跟踪各个寄存器都存放了哪些变量的当前值
地址描述符	某个变量的当前值存放在哪个或哪些位置
getReg函数	根据描述符和数据流信息为三地址指令选择最佳寄存器的函数
DAG (有向无环图)	基本块的图表示，每个变量有对应结点代表初始值
死代码	DAG上没有附加活跃变量的根结点对应的代码
入口结点	流图中额外添加的结点，到第一条指令有一条边
出口结点	流图中额外添加的结点，从可能最后执行的基本块到此有边
前驱/后继	有向边的起点是终点的前驱，终点是起点的后继
循环入口	循环中唯一的、前驱可以在循环外的结点

期末复习七：机器无关优化

名词	解释
代码优化	消除不必要指令，把指令序列替换为更快的指令序列
数据流分析	获取数据沿着程序执行路径流动的信息的技术
数据流抽象	把程序状态集合总结为一些特性的方法
程序点	三地址语句之前或之后的位置
执行路径	从 p_1 到 p_2 的程序点序列
传递函数	基于语句语义的约束， $IN[s] = f_s(OUT[s])$ 或 $OUT[s] = f_s(IN[s])$
控制流方程	基于控制流的约束， $IN[s_{i+1}] = OUT[s_i]$
到达定值	如果存在从定值d后面到达点p的路径且d没被杀死，则d到达p
杀死 (Kill)	路径上对x的其他定值杀死了之前的定值
生成 (Gen)	语句生成了对某个变量的定值
交汇运算符	到达定值中为 $\cup$ ，可用表达式中为 $\cap$
活跃变量分析	分析变量在p上的值是否会在某条从p出发的路径中使用
use集合	在基本块B中引用但引用前没有被定值的集合
def集合	在基本块中定值但定值前没有被引用的变量集合

名词	解释
可用表达式	从入口到p的每条路径都对x+y求值且之后没对x或y赋值
公共子表达式消除	表达式之前已计算过且运算分量值没变，则重用之前结果
复制传播	在x的应用点u可以用y替代x（存在x=y且之后无重新定值）
死代码消除	消除不会影响程序结果的代码
支配结点 (必经节点)	每一条从入口到达n的路径都经过d，则d支配n
后支配	$A \text{ post-dom } B$ ，所有路径从B到Exit都经过A
严格支配	$A \text{ strict-dom } B$ ，但 $A \neq B$
直接支配	$A \text{ strict-dom } B$ ，但不存在C使得 $A \text{ strict-dom } C$ 且 $C \text{ strict-dom } B$
支配结点树	表示支配关系的树，根结点是入口结点
深度优先排序	对流图进行深度优先搜索时的节点排序
DFST (深度优先生成树)	对流图进行深度优先搜索得到的生成树
自然循环	有唯一入口结点支配循环中所有结点的最大强连通分量
循环头	自然循环的唯一入口结点
回边	边 $a \rightarrow b$ ，头b支配了尾a
强连通分量	任意两个节点都互相可达的最大子图
Pre-Header	循环头之前的结点，用于插入代码
最内层循环	不包含其它循环的循环
最小不动点	迭代解法最终收敛到的解
Worklist	迭代算法中维护的工作列表
部分冗余消除	消除在部分路径上重复计算的表达式的优化技术